#25_Pass_by_Reference_vs_Pass_by_Value

11/06/2025

Pass by "value"

- In C, when you "pass by value", the function only has a copy of the original object.
- Operations performed on the object within the function have no effect on the original object outside of the function
- The above can lead you to believe that Python is pass-by-value since reassigning a parameter variable within a function does not affect the variable outside of the function. For example:

```
def change_name(name):
    name = 'Bob'  <-- local scope variable

name = 'Jim'  <-- global scope variable
change_name(name)
print(name) #Output: Jim
```

REASSIGNMENT

The above example appears to demonstrate "pass by value" as reassigning the variable only affected the function-level variable, and not the global variable.

Pass by "reference"

- Python cannot be just pass-by-value, as there would be no way for operations within a function to cause changes to the original object. However you can do this in Python.

11/06/2025

For example:

MUTATION

```
def add_element(my_list):
    my_list.append([4])

my_list = [1, 2, 3]
add_element(my_list)
print(my_list)          # => [1, 2, 3, [4]]
```

The above implies Python is "pass-by-reference", considering operations within the function affected the original object.

However, as was seen in the first example (REASSIGNMENT), not all operations affect the original code.

For example:

REASSIGNMENT

```
def add_element(my_list):
    my_list = my_list + [4]  # [1, 2, 3, 4]

my_list = [1, 2, 3]
add_element(my_list)
print(my_list)          # => [1, 2, 3]
```

The above indicates Python is "pass by value" again.

What Python does

- Python's behaviour is described as "pass by object reference" as you are actually passing a reference to the object (value) rather than a copy of the object itself

11/06/2025

- The function's ability to modify the object depends on whether the object is mutable or not.

- PYTHON ALWAYS PASSES REFERENCES TO OBJECTS, NOT THE OBJECTS THEMSELVES.

- WHEN AN OPERATION WITHIN THE FUNCTION MUTATES THE ARGUMENT, IT WILL AFFECT THE ORIGINAL OBJECT